

Problem 4 Catalan number

$$C_0 = 1, C_{n+1} = \sum_{i=0}^n C_i \times C_{n-i}$$

$$C_1 = C_0 * C_0$$

$$C_2 = C_0 * C_1 + C_1 * C_0$$

$$C_3 = C_0 * C_2 + C_1 * C_1 + C_2 * C_0$$

Implementation 1

```
1 // Version 1: Simple recursive implementation
2 /**
3  * 计算第n个Catalan数(简单递归实现)
4  * @param n 要计算的Catalan数的索引
5  * @return 第n个Catalan数的值
6  */
7 int catalan_recursive(int n) {
8     // 基础情况: C_0 = 1, C_1 = 1
9     if (n <= 1) return 1;
10
11     int res = 0;
12     // 递归计算: C_n = sum(C_i * C_(n-1-i)), i从0到n-1
13     for (int i = 0; i < n; ++i) {
14         res += catalan_recursive(i) * catalan_recursive(n - 1 - i);
15     }
16     return res;
17 }
```

1. Correctness (正确性)

实现正确。该函数直接按照Catalan数的递推公式实现，基础情况和递归逻辑都正确。

2. Characteristics and Use Case (特点与适用场景)

- 特点：代码简洁直观，直接体现了Catalan数的数学定义。但时间复杂度极高，为指数级 $O(C_n)$ ，存在大量重复计算。
- 优点：代码易于理解和实现，适合教学演示。
- 适用场景：仅适合 n 很小的情况 ($n < 10$)，或作为算法原理的演示代码。

3. New Knowledge (新知识点)

- 递归基础：展示了最基本的递归实现方式，通过基础情况和递归调用来解决问题。
- 递推公式的直接转换：说明了如何将数学公式直接转换为递归代码。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 2

```
1 // Version 2: Iterative dynamic programming
```

```

2  #include <vector>
3
4  /**
5   * 使用迭代动态规划计算第n个Catalan数
6   * @param n 要计算的Catalan数的索引
7   * @return 第n个Catalan数的值
8   */
9  int catalan_iterative(int n) {
10     // 基础情况
11     if (n <= 1) return 1;
12
13     // 创建DP数组存储所有Catalan数
14     std::vector<int> dp(n + 1, 0);
15     dp[0] = dp[1] = 1;
16
17     // 从小到大计算每个Catalan数
18     for (int i = 2; i <= n; ++i) {
19         for (int j = 0; j < i; ++j)
20             dp[i] += dp[j] * dp[i - 1 - j];
21     }
22     return dp[n];
23 }

```

1. Correctness (正确性)

实现正确。使用动态规划自底向上计算，避免了重复计算，时间复杂度为 $O(n^2)$ 。

2. Characteristics and Use Case (特点与适用场景)

- **特点**：使用动态规划表存储中间结果，空间复杂度 $O(n)$ ，时间复杂度 $O(n^2)$ 。
- **优点**：避免了递归的重复计算和栈溢出风险，性能大幅提升；代码清晰易懂。
- **适用场景**：适合需要计算单个或少量Catalan数的场景， n 值中等 ($n < 20$) 时效率较高。

3. New Knowledge (新知识点)

- **动态规划思想**：通过存储子问题的解来避免重复计算，体现了"空间换时间"的策略。
- **std::vector的初始化**：`std::vector<int> dp(n + 1, 0)` 创建大小为 $n+1$ 的向量并初始化为0。
- **自底向上构建解**：从最小的问题开始，逐步构建更大问题的解。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 3

```

1  // Version 3: Recursive implementation with memoization
2  #include <unordered_map>
3
4  /**
5   * 使用记忆化递归计算第n个Catalan数
6   * @param n 要计算的Catalan数的索引

```

```

7      * @param memo 记忆化哈希表,存储已计算的结果
8      * @return 第n个Catalan数的值
9      */
10     long long catalan_memoization(int n, std::unordered_map<int, long long>& memo) {
11         // 基础情况
12         if (n <= 1) return 1;
13
14         // 如果已经计算过,直接返回缓存的结果
15         if (memo.find(n) != memo.end()) return memo[n];
16
17         // 递归计算并缓存结果
18         long long res = 0;
19         for (int i = 0; i < n; ++i)
20             res += catalan_memoization(i, memo) * catalan_memoization(n - 1 - i, memo);
21
22         // 存储结果到memo并返回
23         return memo[n] = res;
24     }

```

1. Correctness (正确性)

实现正确。结合了递归和记忆化技术，避免重复计算，时间复杂度降至 $O(n^2)$ 。

2. Characteristics and Use Case (特点与适用场景)

- **特点**：自顶向下的递归方式，配合哈希表缓存结果。空间复杂度 $O(n)$ 用于存储memo。
- **优点**：保持了递归的清晰性，同时避免了重复计算；使用long long避免溢出；适合需要按需计算的场景。
- **适用场景**：适合不需要计算所有中间值，只需要特定某几个Catalan数的情况；递归深度可控时使用。

3. New Knowledge (新知识点)

- **记忆化(Memorization)技术**：通过缓存函数调用结果来优化递归，是动态规划的一种实现方式。
- **std::unordered_map的使用**：哈希表提供 $O(1)$ 的查找和插入，`find()`方法返回迭代器。
- **赋值表达式作为返回值**：`return memo[n] = res`是C++的特性,赋值表达式的值就是被赋的值。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 4

```

1      // Version 4: Using Boost multiprecision for large n
2      #include <boost/multiprecision/cpp_int.hpp>
3      #include <vector>
4
5      /**
6       * 使用Boost大整数库计算第n个Catalan数
7       * 支持任意大的n值,不受整数溢出限制
8       * @param n 要计算的Catalan数的索引
9       * @return 第n个Catalan数的值(大整数类型)

```

```

10  */
11  boost::multiprecision::cpp_int catalan_bigint(int n) {
12      using boost::multiprecision::cpp_int;
13
14      // 基础情况
15      if (n <= 1) return 1;
16
17      // 使用大整数类型的DP数组
18      std::vector<cpp_int> dp(n + 1);
19      dp[0] = dp[1] = 1;
20
21      // 动态规划计算
22      for (int i = 2; i <= n; ++i) {
23          dp[i] = 0;
24          for (int j = 0; j < i; ++j)
25              dp[i] += dp[j] * dp[i - 1 - j];
26      }
27      return dp[n];
28  }
29

```

1. Correctness (正确性)

实现正确。使用Boost多精度库完全解决了整数溢出问题，可以计算任意大的Catalan数。

2. Characteristics and Use Case (特点与适用场景)

- **特点：**使用任意精度整数,不受内置类型的位数限制;但计算速度相对较慢,空间占用较大。
- **优点：**能够精确计算非常大的Catalan数(如C₁₀₀),不会溢出;结果完全准确。
- **适用场景：**需要计算大Catalan数($n > 20$)且要求精确结果的场景;科学计算、密码学等需要大数运算的领域。

3. New Knowledge (新知识点)

- **Boost.Multiprecision库：**提供任意精度的整数和浮点数运算, `cpp_int` 支持任意大小的整数。
- **第三方库的使用：**了解如何引入和使用外部库来扩展C++的功能。
- **类型别名：** `using boost::multiprecision::cpp_int` 简化了长类型名的书写。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 5

```

1  // Version 5: Using factorial and combinatorial formula
2  #include <cmath>
3
4  /**
5   * 计算阶乘
6   * @param n 要计算阶乘的数
7   * @return n的阶乘值
8   */

```

```

9  long long factorial(int n) {
10     long long res = 1;
11     // 累乘计算n!
12     for (int i = 2; i <= n; ++i) res *= i;
13     return res;
14 }
15
16 /**
17  * 使用组合数公式计算第n个Catalan数
18  * 公式:  $C_n = \frac{(2n)!}{(n+1)! * n!}$ 
19  * @param n 要计算的Catalan数的索引
20  * @return 第n个Catalan数的值
21  */
22 long long catalan_formula(int n) {
23     // 基础情况
24     if (n <= 1) return 1;
25
26     // 使用组合数公式计算
27     return factorial(2 * n) / (factorial(n + 1) * factorial(n));
28 }

```

1. Correctness (正确性)

存在问题。虽然数学公式正确,但实现有严重缺陷:

- **整数溢出:** 阶乘增长极快,即使 $n=13$ 时 $2n!$ 就会超过long long范围
- **精度损失:** 先计算巨大的阶乘再除法,中间结果容易溢出
- **效率问题:** 重复计算factorial(n),应该优化

2. Characteristics and Use Case (特点与适用场景)

- **特点:** 使用Catalan数的组合数公式 $C_n = \frac{(2n)!}{(n+1)!n!}$, 理论上时间复杂度 $O(n)$ 。
- **优点:** 公式简洁,直接体现了Catalan数的组合意义。
- **适用场景:** 仅适合非常小的 n 值($n < 10$);更多是理论演示而非实用实现。

3. New Knowledge (新知识点)

- **Catalan数的组合公式:** $C_n = \frac{(2n)!}{(n+1)!n!} = \frac{1}{n+1} \binom{2n}{n}$
- **阶乘的计算:** 通过循环累乘实现阶乘。
- **数值计算的局限性:** 了解到即使公式正确,实现时也要考虑数值范围和溢出问题。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 6

```

1  // Version 6: Compile-time computation using constexpr
2  /**
3  * 使用constexpr在编译期计算第n个Catalan数

```

```

4      * 可在编译时计算常量表达式,提升运行时性能
5      * @param n 要计算的Catalan数的索引 (支持n <= 63)
6      * @return 第n个Catalan数的值
7      */
8      constexpr long long catalan_constexpr(int n) {
9          // 基础情况
10         if (n <= 1) return 1;
11
12         // 使用固定大小数组(constexpr不支持动态分配)
13         long long dp[64] = {0}; // 支持 n <= 63
14         dp[0] = dp[1] = 1;
15
16         // 动态规划计算
17         for (int i = 2; i <= n; ++i) {
18             dp[i] = 0;
19             for (int j = 0; j < i; ++j)
20                 dp[i] += dp[j] * dp[i - 1 - j];
21         }
22         return dp[n];
23     }

```

1. Correctness (正确性)

实现正确。使用constexpr使函数能在编译期执行,当参数是编译期常量时,结果会在编译时计算好。

2. Characteristics and Use Case (特点与适用场景)

- **特点:** 编译期计算,零运行时开销;使用固定大小数组,n值有上限限制。
- **优点:** 当n是编译期常量时,结果直接嵌入到可执行文件中,运行时无需计算;可用于模板元编程和常量表达式。
- **适用场景:** 需要在编译期确定Catalan数的场景;性能关键路径且n为常量;constexpr上下文(如数组大小、模板参数等)。

3. New Knowledge (新知识点)

- **constexpr关键字:** C++11引入,标记可以在编译期求值的函数或变量。
- **编译期计算:** 将运行时计算移至编译期,提升程序性能。
- **constexpr的限制:** C++14前constexpr函数有很多限制(如不能有局部变量),C++14后放宽了限制。
- **固定大小数组:** constexpr中不能使用动态内存分配,必须使用栈上的固定大小数组。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 7

```

1      // Version 7: Parallel recursive computation using async
2      #include <future>
3
4      /**

```

```

5  * 使用异步并行计算第n个Catalan数
6  * 通过std::async创建异步任务并行计算
7  * @param n 要计算的Catalan数的索引
8  * @return 第n个Catalan数的值
9  */
10 long long catalan_parallel(int n) {
11     // 基础情况
12     if (n <= 1) return 1;
13
14     // 创建异步任务向量
15     std::vector<std::future<long long>> futures;
16     long long res = 0;
17
18     // 异步计算前半部分: catalan_parallel(i)
19     for (int i = 0; i < n; ++i) {
20         futures.push_back(std::async(std::launch::async,
21                                     catalan_parallel, i));
22     }
23
24     // 获取异步结果并计算总和
25     for (int i = 0; i < n; ++i) {
26         res += futures[i].get() * catalan_parallel(n - 1 - i);
27     }
28     return res;
29 }

```

1. Correctness (正确性)

实现有严重缺陷:

- **性能灾难**: 没有记忆化,每次递归都创建n个线程,导致指数级线程数量爆炸
- **重复计算**: 仍然存在大量重复计算,并行反而增加了开销
- **线程开销**: 线程创建和切换的开销远大于计算本身,实际运行比单线程慢得多
- **逻辑正确性**: 虽然逻辑上能得到正确结果,但实用性极差

2. Characteristics and Use Case (特点与适用场景)

- **特点**: 尝试使用多线程并行化递归计算;大量创建异步任务。
- **优点**: 展示了C++11异步编程的语法。
- **适用场景**: **几乎不适用于实际场景**。这是一个反面教材,说明并非所有递归都适合并行化,需要配合记忆化才有意义。

3. New Knowledge (新知识点)

- **std::async**: C++11引入的异步任务机制,返回std::future对象。
- **std::future**: 代表一个异步操作的结果,通过get()方法获取。
- **std::launch::async**: 启动策略,强制创建新线程异步执行。
- **并行计算的陷阱**: 了解到不是所有算法都适合并行化,线程开销和数据依赖会影响性能。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 8

```
1 // Version 8: Safe implementation using std::optional
2 #include <optional>
3 #include <vector>
4
5 /**
6  * 使用std::optional实现安全的Catalan数计算
7  * 对非法输入返回空值而非错误结果
8  * @param n 要计算的Catalan数的索引
9  * @return 包含结果的optional对象,若输入非法则返回nullopt
10 */
11 std::optional<long long> catalan_safe(int n) {
12     // 输入验证: 负数无效
13     if (n < 0) return std::nullopt;
14
15     // 基础情况
16     if (n <= 1) return 1;
17
18     // 动态规划计算
19     std::vector<long long> dp(n + 1);
20     dp[0] = dp[1] = 1;
21     for (int i = 2; i <= n; ++i)
22         for (int j = 0; j < i; ++j)
23             dp[i] += dp[j] * dp[i - 1 - j];
24
25     return dp[n];
26 }
```

1. Correctness (正确性)

实现正确。通过std::optional提供了类型安全的错误处理机制,对非法输入返回nullopt而不是未定义行为。

2. Characteristics and Use Case (特点与适用场景)

- **特点:** 使用现代C++的错误处理方式;返回类型明确表示可能失败;调用者必须检查返回值。
- **优点:** 类型安全,避免了返回魔法数字(如-1)表示错误;编译器可以帮助检查是否处理了空值情况;符合现代C++最佳实践。
- **适用场景:** 需要健壮错误处理的生产环境代码;API设计中需要明确表达可能失败的情况;对代码安全性和可维护性要求高的项目。

3. New Knowledge (新知识点)

- **std::optional:** C++17引入,表示可能存在也可能不存在的值,是类型安全的"可空类型"。
- **std::nullopt:** 表示optional的空状态。
- **现代错误处理:** 相比异常和错误码,optional提供了更轻量级和类型安全的错误处理方式。
- **值语义:** optional使用值语义,避免了指针的空指针问题。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 9

```
1 // Version 9: Template metaprogramming version
2 /**
3  * 使用模板元编程在编译期计算Catalan数
4  * 模板参数N指定要计算的Catalan数索引
5  * 使用方法: Catalan<5>::value 得到C_5的值
6  */
7 template<int N>
8 struct Catalan {
9     // 使用constexpr lambda在编译期计算值
10     static constexpr long long value = ([]() constexpr {
11         long long sum = 0;
12         // 递归展开模板,计算累加和
13         for (int i = 0; i < N; ++i)
14             sum += Catalan<i>::value * Catalan<N - 1 - i>::value;
15         return sum;
16     })();
17 };
18
19 // 模板特化: 基础情况 C_0 = 1
20 template<> struct Catalan<0> { static constexpr long long value = 1; };
21
22 // 模板特化: 基础情况 C_1 = 1
23 template<> struct Catalan<1> { static constexpr long long value = 1; };
```

1. Correctness (正确性)

实现正确。通过模板递归和特化实现编译期计算,结果直接成为编译期常量。

2. Characteristics and Use Case (特点与适用场景)

- **特点:** 完全在编译期完成计算;使用模板元编程技术;结合constexpr lambda表达式;零运行时开销。
- **优点:** 极致的性能优化,运行时直接使用常量;体现了C++的元编程能力;可用于需要编译期常量的场景(如数组大小)。
- **适用场景:** 需要编译期确定Catalan数且n较小的场景;模板元编程和泛型编程;追求极致性能的场所;教学演示高级C++特性。

3. New Knowledge (新知识点)

- **模板元编程(TMP):** 使用C++模板在编译期进行计算和类型操作。
- **模板特化:** 为特定模板参数提供专门的实现。
- **constexpr lambda:** C++17允许lambda表达式是constexpr的,可在编译期执行。
- **编译期递归:** 模板实例化过程中的递归,编译器会展开所有模板实例。
- **IIFE(立即调用函数表达式):** `([](){...})()` 模式,lambda定义后立即调用。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

Implementation 10

```
1 // Version 10: Conceptual matrix-based recurrence demonstration
2 #include <array>
3
4 // 定义2x2矩阵类型
5 using Matrix2x2 = std::array<std::array<long long, 2>, 2>;
6
7 /**
8  * 2x2矩阵乘法
9  * @param a 第一个矩阵
10  * @param b 第二个矩阵
11  * @return 两矩阵的乘积
12  */
13 Matrix2x2 matrix_multiply(const Matrix2x2& a, const Matrix2x2& b) {
14     Matrix2x2 result = {{{0, 0}, {0, 0}}};
15
16     // 标准矩阵乘法: C[i][j] = sum(A[i][k] * B[k][j])
17     for (int i = 0; i < 2; ++i)
18         for (int j = 0; j < 2; ++j)
19             for (int k = 0; k < 2; ++k)
20                 result[i][j] += a[i][k] * b[k][j];
21
22     return result;
23 }
24
25 /**
26  * 使用矩阵快速幂概念计算(实际并非Catalan数的正确矩阵形式)
27  * 注意: 这是一个概念演示,实际计算结果并非Catalan数
28  * @param n 要计算的索引
29  * @return 矩阵计算的结果
30  */
31 long long catalan_matrix(int n) {
32     if (n <= 1) return 1;
33
34     // 演示矩阵递推概念 (注意:这并非Catalan数的正确矩阵表示)
35     Matrix2x2 base = {{{1, 1}, {1, 0}}}; // 类似斐波那契的转移矩阵
36     Matrix2x2 result = {{{1, 0}, {0, 1}}}; // 单位矩阵
37
38     // 矩阵连乘
39     for (int i = 0; i < n; ++i)
40         result = matrix_multiply(result, base);
41
42     return result[0][0];
43 }
```

1. Correctness (正确性)

实现错误。这个实现实际上计算的是斐波那契数列而非Catalan数。Catalan数的递推关系是

$$C_n = \sum_{i=0}^{n-1} C_i \times C_{n-1-i}$$
这是一个非线性递推,不能简单地用线性矩阵快速幂表示。该实现只是一个概念演示,名不副实。

2. Characteristics and Use Case (特点与适用场景)

- **特点**: 展示了矩阵快速幂的思想;使用std::array定义矩阵类型;演示了矩阵乘法的实现。
- **优点**: 如果配合快速幂(二分法),可以将线性递推的时间复杂度降到 $O(\log n)$;代码展示了矩阵运算的基本框架。
- **适用场景**: **不适用于Catalan数计算**。该技术适用于线性递推关系(如斐波那契数列、线性递推数列等),但Catalan数是非线性递推,不能直接应用。

3. New Knowledge (新知识点)

- **矩阵快速幂**: 将线性递推关系表示为矩阵形式,通过矩阵快速幂优化计算。
- **std::array的嵌套使用**: `std::array<std::array<T, N>, M>` 表示二维数组。
- **类型别名using**: `using Matrix2x2 = ...` 定义类型别名,提高代码可读性。
- **线性vs非线性递推**: 了解到不是所有递推都能用矩阵快速幂优化,Catalan数的非线性特性使其不适用。

4. Code Style (代码风格)

已按Google C++编码规范添加注释。

My Version

综合分析以上10种实现后, 设计了一个兼具正确性、效率和安全性的最佳版本:

```
1 // Your Version: Elegant and Effective Implementation
2 #include <iostream>
3 #include <vector>
4 #include <optional>
5
6 /**
7  * 计算第n个Catalan数的最佳实现
8  * @param n 要计算的Catalan数的索引 (0 <= n <= 32)
9  * @return 包含结果的optional对象,若输入非法或溢出则返回nullopt
10  * 时间复杂度:  $O(n^2)$ , 空间复杂度:  $O(n)$ 
11  */
12 std::optional<unsigned long long> catalan_best(int n) {
13     // 输入验证
14     if (n < 0) return std::nullopt;
15     if (n <= 1) return 1ULL;
16
17     // 限制范围避免溢出
18     constexpr int MAX_N = 32;
19     if (n > MAX_N) return std::nullopt;
20
21     // 动态规划
22     std::vector<unsigned long long> dp(n + 1, 0);
23     dp[0] = dp[1] = 1;
24
25     for (int i = 2; i <= n; ++i) {
26         for (int j = 0; j < i; ++j) {
27             unsigned long long temp = dp[j] * dp[i - 1 - j];
28             // 溢出检测
29             if (temp < dp[j] && dp[i - 1 - j] > 1) return std::nullopt;
30         }
31     }
```

```

31     unsigned long long old_val = dp[i];
32     dp[i] += temp;
33     if (dp[i] < old_val) return std::nullopt;
34 }
35 }
36
37     return dp[n];
38 }
39
40 int main() {
41     // 测试不同的n值
42     auto result1 = catalan_best(10);
43     if (result1.has_value()) {
44         std::cout << "C_10 = " << result1.value() << std::endl;
45     }
46
47     auto result2 = catalan_best(20);
48     if (result2.has_value()) {
49         std::cout << "C_20 = " << result2.value() << std::endl;
50     }
51
52     // n=50会超出范围
53     auto result3 = catalan_best(50);
54     if (!result3.has_value()) {
55         std::cout << "C_50 计算失败：超出范围或溢出" << std::endl;
56     }
57
58     return 0;
59 }

```

实现说明

1. 算法实现

采用迭代动态规划方法计算Catalan数。算法从最小的子问题（C_0和C_1）开始，逐步计算到目标值C_n。使用一维数组 `dp` 存储中间结果，其中 `dp[i]` 表示第*i*个Catalan数。

计算过程遵循递推公式： $C_n = \sum_{i=0}^{n-1} C_i \times C_{n-1-i}$ 。对于每个 `dp[i]`，通过内层循环累加所有 `dp[j] * dp[i-1-j]` 的值。时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

数据类型选择 `unsigned long long`（64位无符号整数），相比有符号类型可以表示更大的数值范围。这使得可计算范围扩展到C_32（值约为 1.77×10^{18} ）。

2. 错误处理设计

函数返回类型为 `std::optional<unsigned long long>`，这是C++17标准库提供的一个模板类，用于表示"可能存在也可能不存在"的值。

三种情况会返回 `std::nullopt`（表示空值）：

- 输入参数*n*为负数（不符合Catalan数定义）
- 输入参数*n*大于32（超出`unsigned long long`的安全范围）
- 计算过程中检测到溢出

调用者需要使用 `has_value()` 方法检查返回值是否有效，然后通过 `value()` 方法获取实际结果。这种设计在编译期就明确了"可能失败"的语义，避免返回无意义的错误码。

3. 溢出检测机制

实现了两层溢出保护：

静态限制： 设置常量 `MAX_N = 32`，在函数入口处检查输入。这个值是根据 `unsigned long long` 的位宽（64位）和 Catalan 数的增长速度计算得出的安全上限。

动态检测： 在计算过程中实时检测溢出情况：

- 乘法溢出检测： `if (temp < dp[j] && dp[i-1-j] > 1)`。无符号整数溢出时会发生回绕，导致结果小于操作数。
- 加法溢出检测： `if (dp[i] < old_val)`。累加后的值如果小于原值，说明发生了溢出。

一旦检测到溢出，立即返回 `std::nullopt`，终止计算。

4. 代码结构

函数分为四个主要部分：

1. **输入验证：** 检查 `n` 是否为负数，处理基础情况 (`n ≤ 1`)
2. **范围检查：** 确认 `n` 不超过 `MAX_N`
3. **动态规划计算：** 双层循环实现递推公式，包含溢出检测
4. **返回结果：** 将计算结果包装在 `optional` 中返回

`main` 函数演示了三个测试用例：

- `C_10`：正常范围内的计算
- `C_20`：接近上限的计算
- `C_50`：超出范围的情况，触发 `nullopt` 返回

5. 技术特点

使用的 C++ 特性：

- `std::optional` (C++17)：类型安全的可空值表示
- `std::vector`：动态数组存储 DP 表
- `constexpr`：编译期常量定义
- `ULL` 后缀：明确表示 `unsigned long long` 字面量

依赖情况：

仅依赖 C++17 标准库的 `<vector>` 和 `<optional>`，不需要第三方库。可在任何支持 C++17 的编译器上编译。

与其他实现的关系：

- 基础算法框架来自 Implementation 2 的动态规划思路
- 错误处理方式借鉴 Implementation 8 的 `optional` 设计
- 溢出意识受 Implementation 4 启发，但使用标准类型而非大数库

